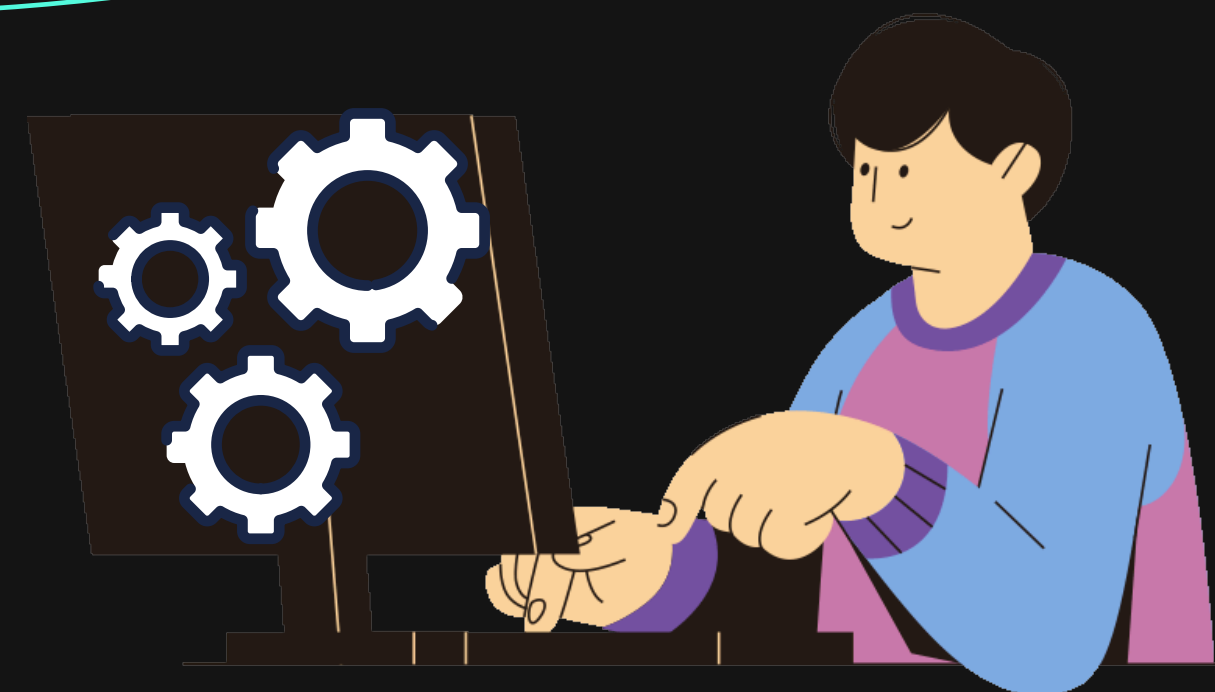


Classificação Utilizando Redes Neurais Multicamadas

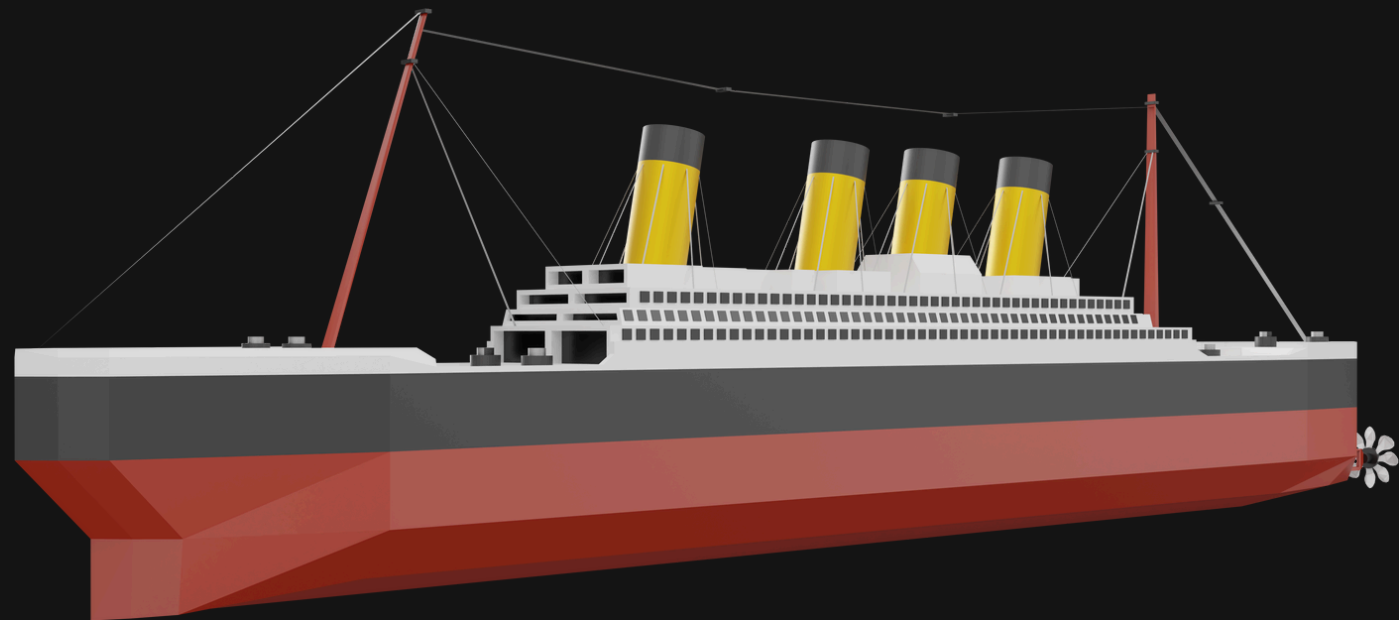
TÓPICOS EM APRENDIZADO DE MÁQUINA

- PEDRO HENRIQUE GURSKI DE OLIVEIRA
- ULISSES CURVELHO



Datasets Utilizados

- **Titanic;** (Sobreviveram/Não Sobreviveram)
- **Depressão estudantil;** (Com depressão/Sem depressão)
- **Avaliação de carros.** (não-aceito, aceito, bom, muito bom)



Tratamento de Dados

PassengerId	# Survived	# Pclass	A Name	A Sex	# Age	# SibSp	# Parch	A Ticket	# Fare
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1
5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05
6	0	3	Moran, Mr. James	male		0	0	330877	8.4583
7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625
8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1	349909	21.075
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333
10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14	1	0	237736	30.0708
11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549	16.7
12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0	113783	26.55

Tratamento de Dados

Melhorias Dataset:

- Remoção colunas inviáveis;
- Valores Nulos: Fazemos a média ou mediana;
- Codificação de variáveis categóricas;
- StandardScaler (Padronização).

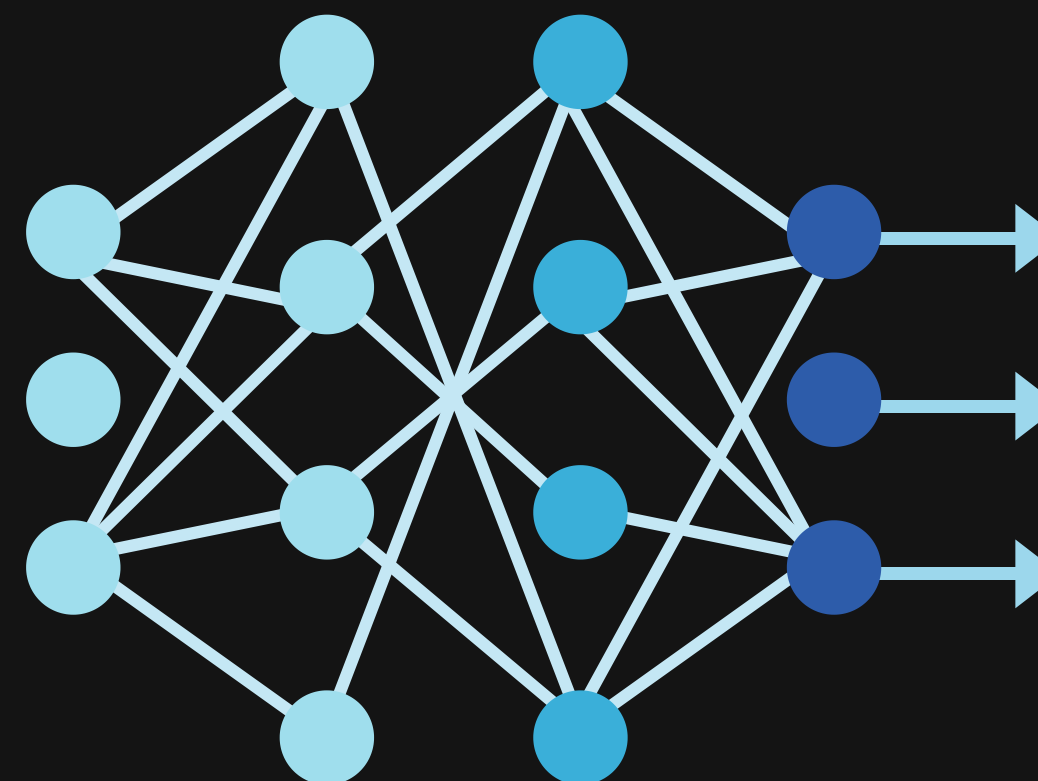
PassengerId	# Survived	# Pclass	A Name	A Sex	# Age	# SibSp	# Parch	A Ticket	# Fare
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25
2	1	1	Cumings, Mrs. John Bradley	female	38	1	0	PC 17599	71.2833
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925
4	1	1	Jacques Heath (Lily May Peel)	female	35	0	0	113803	53.1
5	0	3	Moran, Mr. James	male	30	0	0	330877	8.05
6	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625
7	0	3	Gossard, Mr. Leonard	male	20	0	0	3101	21.075
8	0	3	Berg, Miss. Vilhelmina	female	22	0	0	347742	11.1333
9	0	3	Nasser, Mrs. Nicholas (Adele Achem)	female	14	1	0	237736	30.0708
10	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549	16.7
11	1	1	Bonnell, Miss. Elizabeth	female	58	0	0	113783	26.55
12	0	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925

Desenvolvimento do Modelo

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim=64):
        super(MLP, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 10) # número máximo de classes
        )

    def forward(self, x):
        return self.model(x)
```

• 3 CAMADAS
• 64 NEURÔNIOS POR CAMADA



Desenvolvimento do Modelo

Validação Cruzada k-Fold

Dividimos todo o conjunto de dados em k partes de mesmo tamanho (ou o mais próximo possível).

Em cada uma das k iterações (folds):

- 1 parte é usada para teste
- k - 1 partes são usadas para treino

Usa todo o dataset para treino e teste em diferentes rodadas.

Titanic

Fold 1/5
Acurácia: 0.8212 , Precisão: 0.8230 Recall: 0.8212 | F1-score: 0.8182

Fold 2/5
Acurácia: 0.8090 Precisão: 0.8108 Recall: 0.8090 | F1-score: 0.8025

Fold 3/5
Acurácia: 0.8652 Precisão: 0.8667 Recall: 0.8652 | F1-score: 0.8630

Fold 4/5
Acurácia: 0.7921 Precisão: 0.8076 Recall: 0.7921 | F1-score: 0.7766

Fold 5/5
Acurácia: 0.8258 Precisão: 0.8260 Recall: 0.8258 | F1-score: 0.8209

```
def cross_validate(X_tensor, y_tensor, k=5, hidden_dim=64):
    kf = KFold(n_splits=k, shuffle=True, random_state=42)

    accs, precs, recs, f1s = [], [], [], []
    # Número de classes diferentes no alvo
    output_dim = len(torch.unique(y_tensor))

    # Loop por cada fold da validação cruzada
    for fold, (train_idx, test_idx) in enumerate(kf.split(X_tensor)):
        print(f"\nFold {fold+1}/{k}")

        # Separa os dados de treino e teste com base nos índices do fold
        X_train, y_train = X_tensor[train_idx], y_tensor[train_idx]
        X_test, y_test = X_tensor[test_idx], y_tensor[test_idx]

        # Cria datasets e dataloaders para treino e teste
        train_ds = TensorDataset(X_train, y_train)
        test_ds = TensorDataset(X_test, y_test)

        train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
        test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)

        model = MLP(X_tensor.shape[1], hidden_dim)
        # Treina e avalia o modelo nesse fold, retornando métricas
        acc, prec, rec, f1 = train_and_evaluate(model, train_loader, test_loader, output_dim)
        accs.append(acc); precs.append(prec); recs.append(rec); f1s.append(f1)

    print("\nMÉDIAS GERAIS:")
    print(f"Acurácia Média: {np.mean(accs):.4f}")
    print(f"Precisão Média: {np.mean(precs):.4f}")
    print(f"Recall Médio: {np.mean(recs):.4f}")
    print(f"F1-score Médio: {np.mean(f1s):.4f}")
```

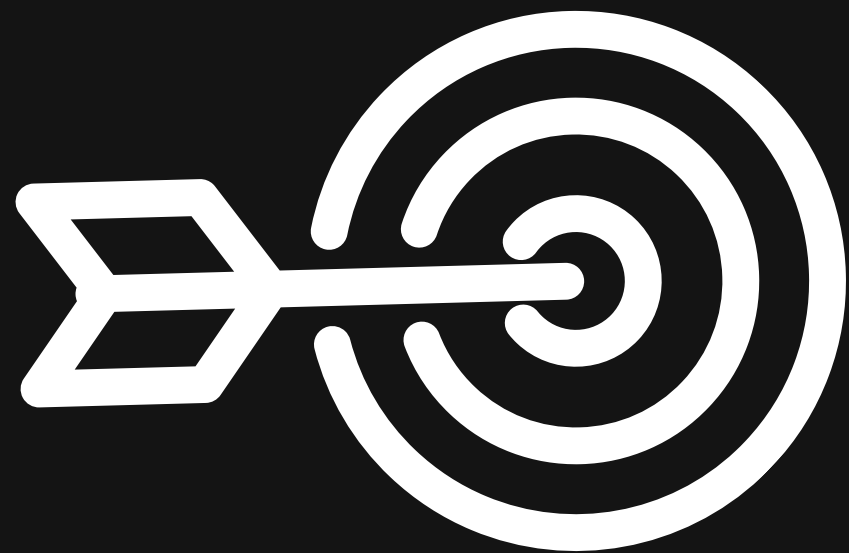
RESULTADOS

Métrica	Acurácia	Precisão	Recall	F1-score
Titanic	0.8193	0.8233	0.8193	0.8128
Depressão	0.8453	0.8449	0.8453	0.8448
Carro	0.9218	0.9085	0.9218	0.9093

OBS: Por o Dataset Carro ser desbalanceado, unacc (70.023 %), acc (22.222 %), good (3.993 %) e v-good (3.762 %) — pode ser enganosa se o modelo apenas "chutar" a classe majoritária.

Desafios enfrentados

- Dificuldade ao encontrar modelos viáveis para o modelo;
- Identificar melhorias dos datasets para um melhor desempenho;
- Melhor forma para se treinar os modelos;



Melhorias

- Datasets mais robustos e tratamento exclusivo;
- Batch Normalization;
- Explorar outras funções de ativação(LeakyReLU, GELU, SiLU ...);
- Early Stopping;



